

MODULE 3: PROGRAMMING FOR BIOLOGICAL DATA

Visualization & Reporting

ggtree and R Markdown

Instructor: Dr. Haogao Gu | **Date:** February 11, 2026

Welcome to the Final Session! 🎉

What You've Accomplished

- **Sessions 1-4:** R Fundamentals (Data structures, dplyr)
- **Sessions 5-8:** Statistics & Quality Control (t-tests, ANOVA, QC charts)
- **Sessions 9-11:** Bioinformatics (Sequences, MSA, Trees)



Today's Goal

Learn to visualize complex data professionally and generate reproducible reports.

Learning Objectives



ggtree

Create publication-quality tree figures.



Metadata

Annotate trees with clinical data (colors, labels).



Export

Save figures as high-resolution PNG/PDF.



R Markdown

Understand reproducible reporting concepts.

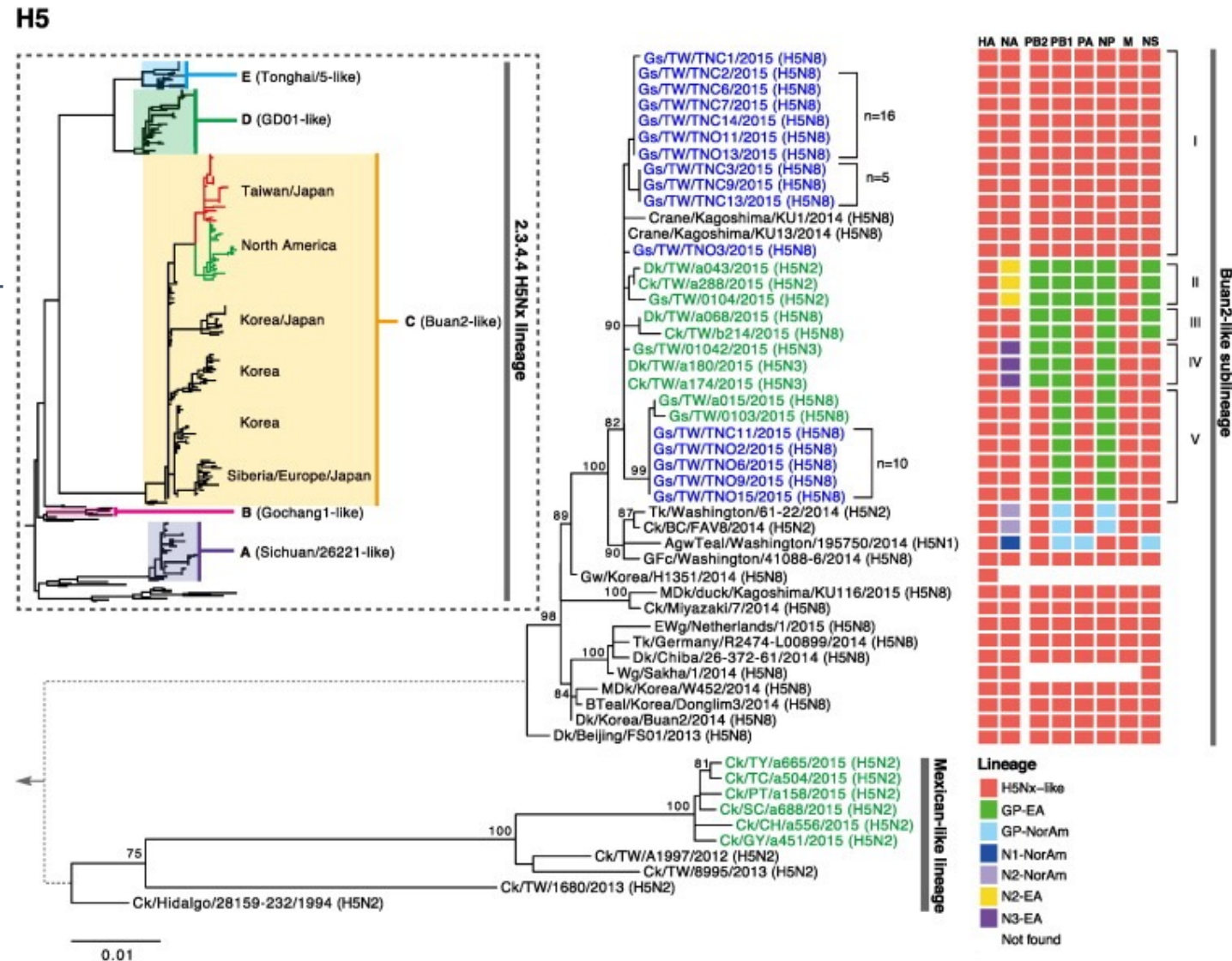
Part 1: Advanced Visualization with ggtree

Turning Data into Insight

What is ggtree?

A **Bioconductor package** extending ggplot2 for phylogenetic trees.

- Beautiful, publication-ready aesthetics.
- Follows standard "Grammar of Graphics".
- Easy layer addition (+ geom_...).
- Flexible layouts (rectangular, circular, fan).



ggtree vs. Base R (ape)

Feature	ape plot()	ggtree
Appearance	Basic, functional	Polished, customizable
Customization	Limited parameters	Unlimited layers
Metadata	Manual mapping	Built-in support (%<+%)
Learning Curve	Low	Medium (ggplot style)

Basic ggtree Syntax

```
library(ggtree)

# The simplest tree plot
ggtree(tree) +
  geom_tiplab()
```

Just like ggplot(), we initialize the object and add geometric layers.

Adding Layers

Code

```
ggtree(tree) +  
  geom_tiplab() +  
  geom_tippoint() +  
  geom_nodepoint() +  
  geom_treescale()
```

Explanation

- `geom_tiplab()`: Adds text labels to tips.
- `geom_tippoint()`: Adds dots to tips.
- `geom_nodepoint()`: Adds dots to internal nodes.
- `geom_treescale()`: Adds a distance scale bar.

Tree Layouts

```
# Rectangular (Standard) ggtree(tree, layout = "rectangular") # Circular (Compact for large trees)
ggtree(tree, layout = "circular") # Fan (Spreads out tips) ggtree(tree, layout = "fan")
```

Attaching Metadata

The %<+% operator allows linking a data frame to the tree.

```
# Metadata frame
# Must have a column matching tip labels
metadata <- data.frame(
  label = c("Sample1", "Sample2"),
  Location = c("HK", "SZ")
)

# Attach and plot
ggtree(tree) %<+% metadata +
  geom_tiplab(aes(color = Location))
```

Coloring by Metadata

```
ggtree(tree) %<+% metadata +  
  geom_tiplab(aes(color = Location)) +  
  scale_color_brewer(palette = "Set1")
```

Any ggplot2 color scale works here!

Adding a Scale Bar

```
ggtree(tree) +  
  geom_tiplab() +  
  geom_treescale(x = 0, y = 0) +  
  theme_tree2()
```

geom_treescale(): Adds a floating scale bar.

theme_tree2(): Adds an X-axis with distance values.

Highlighting Clades

Emphasize specific clusters (e.g., an outbreak group).

```
ggtree(tree) +  
  geom_tiplab() +  
  geom_highlight(node = 8,  
                 fill = "yellow",  
                 alpha = 0.3)
```

Requires knowing the internal **node ID**.

Circular Trees

```
ggtree(tree, layout = "circular") +  
  geom_tiplab(aes(angle = angle),  
             hjust = -0.1) +  
  geom_tippoint(aes(color = Location))
```

Great for visualizing trees with many tips (50+).

Saving with ggsave()

```
# Assign plot to variable p <- ggtree(tree) + geom_tiplab() # Save as PNG (Raster) ggsave("tree.png", p,  
width = 8, height = 6) # Save as PDF (Vector) ggsave("tree.pdf", p, width = 8, height = 6)
```

Publication Tips

- ✓ Use `theme_tree2()` for clear axes.
- ✓ Set appropriate dimensions to avoid text overlap.
- ✓ Use `dpi = 300` for print quality.
- ✓ Choose colorblind-friendly palettes.
- ✓ Add clear legends.

Part 2: Local R Development

Moving Beyond the Cloud

Beyond Google Colab

Colab Limitations

- Disconnects after ~12 hours.
- Files lost when runtime recycles.
- Limited computing power.

Local Development

- Persistent files.
- Full power of your machine.
- Secure data storage.
- Standard in professional settings.

The RStudio IDE

The standard professional interface for R.

1

Source

Write scripts here.

2

Console

Run code
immediately.

3

Environment

See your data.

4

Files/Plots

Manage outputs.

The "R Project" (.Rproj)

The Problem

```
setwd("C:/Users/Haogao/Downloads/...")
```

This code breaks on anyone else's computer.

The Solution

R Projects set the working directory to the project folder automatically.

Portable: Send the folder to anyone, it just works!

Standard Directory Structure

Organize your work like a pro:

```
My_Project/  
├── My_Project.Rproj  
├── data/          ← Raw input files (Read-only)  
├── scripts/       ← Analysis code (.R)  
└── results/       ← Output plots & tables
```

Golden Rule: Keep inputs separate from outputs.

Scripts (.R) vs Notebooks (.ipynb)

Feature	Notebooks	Scripts
Use Case	Exploration, Teaching	Pipelines, Automation
Output	Inline HTML/PDF	Saved Files (PNG/CSV)
Execution	Cell by Cell	<code>source("analysis.R")</code>
Version Control	Difficult (JSON)	Easy (Plain Text)

Managing File Paths

Absolute (BAD)

```
"/Users/haogao/Documents/Project/data/file.csv"
```

Only works on one computer.

Relative (GOOD)

```
"data/file.csv"
```

Works anywhere inside the project folder.

Running Scripts

You can automate an entire analysis with one command:

```
# Run data cleaning  
source("scripts/01_clean_data.R")  
  
# Run tree building  
source("scripts/02_make_tree.R")
```

This ensures reproducibility from start to finish.

Installing Packages Locally

In Colab: You install every time.

Locally:

1. Run `install.packages("ggtree")` **ONCE**.
2. Use `library(ggtree)` in your scripts.

Libraries stay installed on your hard drive.

Key Takeaways

- **ggtree** provides advanced visualization power.
- **RStudio + R Projects** is the professional standard environment.
- **Relative Paths** ensure portability.
- **Scripts (.R)** enable reproducible automation.



Course Complete!

From Variables to Visualizations.

You are now R-Ready for Biological Data.

Tutorial 12

Setting Up Your Pipeline

Open your R environment

Tutorial 12: Setting Up Your Pipeline

Automating Analysis with Scripts

Your Scenario

The Task

You have finished an analysis in a notebook. Now, formalize it into a **Reproducible Pipeline**.

Steps:

1. Set up directories.
2. Create dummy data.
3. Write an R script.
4. Execute the pipeline.

Step 1: Project Structure

Use `dir.create()` to make the folders.

```
# TODO: Fill in names
dir.create(" ... ") # Data
dir.create(" ... ") # Scripts
dir.create(" ... ") # Results
```

Verify with `list.dirs()`.

Step 2: Create Data

Create a simple CSV file in the data folder.

```
my_data <- data.frame( ... )  
  
write.csv(my_data,  
          "my_data/experiment.csv",  
          row.names = FALSE)
```


Step 3: Write Script

We will programmatically create a script file.

```
script_content ← '  
library(ggplot2)  
df ← read.csv("my_data/experiment.csv")  
p ← ggplot(df, aes(x=Sample, y=Value)) + geom_col()  
ggsave("my_results/plot.png", p)  
'  
  
writeLines(script_content, "my_scripts/run.R")
```

Step 4: Execute Pipeline

Run the script using source().

```
# TODO: Run it
source(" ... ")

# Check result
file.exists("my_results/plot.png")
```

If true, you have successfully built an automated pipeline!

Session 12 Complete

Congratulations on completing the module!